

一、整合概述

libmnl 是一个面向 Netlink 开发者的极简用户空间库，提供了一组函数和宏来简化 Netlink 应用程序的开发。本文档记录了 libmnl 在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- libmnl 版本：1.0.5
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux
- 目标架构：ARM64 (aarch64)
- 编译日期：2026-05-18

二、整合步骤

1. 产品配置修改

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/config.json`

在 `thirdparty` 子系统的 `components` 数组中添加 libmnl 组件：

修改前：

```
{
  subsystem: thirdparty,
  components: [
    { component: zlib, features:[] },
    { component: openssl, features:[] },
    { component: busybox, features:[] }
  ]
}
```

修改后：

```
{
  subsystem: thirdparty,
  components: [
    { component: zlib, features:[] },
    { component: openssl, features:[] },
    { component: libmnl, features:[] },
    { component: busybox, features:[] }
  ]
}
```

注意： 若后续需添加 ethtool，libmnl 必须在 ethtool 之前，因为 ethtool 依赖 libmnl。

2. 构建脚本说明（无需修改）

文件: `third_party/libmnl/BUILD.gn`

BUILD.gn 已针对 lite 和 standard 两种系统类型做了分支处理，无需额外修改。hispark_hi3519dv500_linux 属于 small 类型，走 ohos_lite 分支：

```
if (defined(ohos_lite)) {
  lite_library("libmnl_shared") {
    target_type = "shared_library"
    sources = libmnl_SRC
    include_dirs = common_include_dirs
    cflags = [
      "-Wall", "-Wextra", "-Werror", "-g", "-O2",
      "-DHAVE_CONFIG_H", "-fvisibility=hidden",
      "-Wno-unused-parameter",
      "-Wno-gnu-variable-sized-type-not-at-end",
      "-Wno-unused-but-set-variable", "-fPIC"
    ]
    public_configs = [ ":libmnl_public_config" ]
    deps = [ "//third_party/musl:sysroot_lite" ]
  }
}
```

3. 组件描述文件（无需修改）

文件: `third_party/libmnl/bundle.json`

```
{
  "name": "@ohos/libmnl",
  "component": {
    "name": "libmnl",
    "subsystem": "thirdparty",
    "adapted_system_type": ["small", "standard"],
    "build": {
      "sub_component": [
        "//third_party/libmnl:libmnl_shared"
      ]
    }
  }
}
```

4. 头文件适配（已完成）

文件: `third_party/libmnl/include/libmnl/libmnl.h`

头文件已做 OpenHarmony 兼容处理，使用系统 netlink 头文件替代本地版本以避免冲突：

```
#ifndef __kernel_sa_family_t
#define __kernel_sa_family_t sa_family_t
#endif
#define __UAPI_DEF_SOCKADDR_STORAGE 0
#include <linux/netlink.h>
```

三、编译验证

编译命令

```
# 单独编译 libmnl
hb build //third_party/libmnl:libmnl_shared
```

编译结果

```
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:01:36
```

生成文件

```
# 共享库 (stripped)
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/usr/lib/libmnl_shared.so

# 未 strip 的版本 (包含调试符号)
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/unstripped/usr/lib/libmnl_shared.so

# rootfs 中的版本 (随镜像部署)
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/rootfs/usr/lib/libmnl_shared.so
```

文件信息

```
$ ls -lh out/.../usr/lib/libmnl_shared.so
-rwxrwxr-x 1 chenzhy223 chenzhy223 26K 5月 18 00:38 libmnl_shared.so

$ file out/.../usr/lib/libmnl_shared.so
libmnl_shared.so: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically
linked, stripped
```

文件特性:

- 文件大小: 26KB (stripped)
- 架构: ARM 64-bit (aarch64)
- 链接方式: 动态链接共享库
- 编译工具链: aarch64-v01c01-linux-musl-gcc

符号导出验证

```
$ nm -D out/.../unstripped/usr/lib/libmnl_shared.so | grep -c ' T mnl'
69
```

导出 69 个 `mnl_*` 函数，涵盖四大模块：

模块	主要函数
attr (属性操作)	<code>mnl_attr_get_type</code> , <code>mnl_attr_get_len</code> , <code>mnl_attr_get_payload</code> , <code>mnl_attr_put</code> , <code>mnl_attr_parse</code> , <code>mnl_attr_nest_start</code> 等
callback (回调处理)	<code>mnl_cb_run</code> , <code>mnl_cb_run2</code>
nlmsg (消息操作)	<code>mnl_nlmsg_put_header</code> , <code>mnl_nlmsg_put_extra_header</code> , <code>mnl_nlmsg_batch_start</code> , <code>mnl_nlmsg_next</code> 等
socket (套接字)	<code>mnl_socket_open</code> , <code>mnl_socket_close</code> , <code>mnl_socket_bind</code> , <code>mnl_socket_sendto</code> , <code>mnl_socket_recvfrom</code> 等

四、功能说明

libmnl 为用户空间程序提供了与 Linux 内核 Netlink 接口通信的能力，主要功能包括：

Netlink Socket 管理

- `mnl_socket_open()` - 打开 Netlink 套接字
- `mnl_socket_close()` - 关闭套接字
- `mnl_socket_bind()` - 绑定套接字
- `mnl_socket_sendto()` / `mnl_socket_recvfrom()` - 收发消息

Netlink 消息构造与解析

- `mnl_nlmsg_put_header()` - 构建消息头
- `mnl_nlmsg_put_extra_header()` - 添加额外头部
- `mnl_nlmsg_get_payload()` - 获取消息载荷

Netlink 属性操作

- `mnl_attr_parse()` - 解析属性链
- `mnl_attr_get_u32()` / `mnl_attr_put_u32()` - 读写 32 位属性
- `mnl_attr_nest_start()` / `mnl_attr_nest_end()` - 嵌套属性

回调机制

- `mn1_cb_run()` - 消息回调处理

五、使用方法

1. 编译时链接

在 BUILD.gn 中添加对 libmn1 的依赖:

```
deps = [ "//third_party/libmn1:libmn1_shared" ]
```

2. 代码中引用头文件

```
#include <libmn1/libmn1.h>
```

3. 运行时部署

将 `libmn1_shared.so` 推送到设备:

```
hdc file send libmn1_shared.so /usr/lib/
```

4. 代码示例：打开 Netlink Socket

```
#include <stdio.h>
#include <libmn1/libmn1.h>

int main(void) {
    struct mn1_socket *n1;

    n1 = mn1_socket_open(NETLINK_ROUTE);
    if (n1 == NULL) {
        perror("mn1_socket_open");
        return 1;
    }

    printf("libmn1 socket opened successfully\n");
    mn1_socket_close(n1);
    return 0;
}
```

六、与其他组件的关系

依赖关系

- **musl libc**: libmnl 依赖 musl C 标准库 (`sysroot_lite`)
- **Linux kernel headers**: 需要内核头文件 (netlink, socket 等)
- 无外部第三方依赖

被依赖关系

- **ethtool**: 依赖 libmnl 进行 Netlink 通信, config.json 中 libmnl 必须在 ethtool 之前

配合使用示例

```
# ethtool 通过 libmnl 查询网卡信息
ethtool eth0

# 搭配 busybox 的网络工具
busybox ifconfig -a
ethtool -i eth0
```

七、注意事项

1. 架构差异

本产品 (hispark_hi3519dv500_linux) 使用 ARM64 架构:

- 编译工具链: `aarch64-v01c01-linux-musl-gcc`
- 动态链接器: `/lib/ld-musl-aarch64.so.1`
- 与 hispark_taurus_linux (ARM 32-bit) 的编译工具链不同

2. 系统类型

hispark_hi3519dv500_linux 属于 `small` 类型系统, BUILD.gn 中走 `ohos_lite` 分支。若用于 `standard` 类型系统, 则走 `ohos_shared_library` 分支。

3. 组件顺序

若后续添加 ethtool, config.json 中 libmnl 必须在 ethtool 之前声明, 因为构建系统按顺序解析依赖。

4. 库文件名

编译产物为 `libmnl_shared.so` (非标准的 `libmnl.so`), 运行时链接需注意库名。

5. 安装路径

共享库安装到 `/usr/lib/` 目录，随 system 镜像部署。运行时需确保 `LD_LIBRARY_PATH` 包含此路径。

八、参考资料

- libmnl 官网: <https://www.netfilter.org/projects/libmnl/>
- Netlink 协议文档: <https://man7.org/linux/man-pages/man7/netlink.7.html>
- OpenHarmony 构建系统文档: <https://gitee.com/openharmony>